📚 Stack – Data Structure

1. Introduction to Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle: the element inserted last is accessed first.

> 🔹 **Key concept:** LIFO — Last In, First Out.

Real-life examples

- Stack of plates in a cafeteria
- Stack of books
- Browser back button (most recent page is at the top)

---

2. Operations on Stack

Primary Operations

- **Push** – adds an element at the **top** of the stack.
- **Pop** – removes the **topmost** element and returns it.

Secondary Operations

- **isEmpty** – checks if the stack is empty, returns `True` or `False`.
- **Peek / Display** – views the top element without removing it; can display all elements without deletion.

---

3. Implementation using List in Python

Python lists are used to implement stacks efficiently.

```
# Initialise an empty stack
stack = []
```

Push operation

```
def push(stack, item):
    stack.append(item)      # adds at the end (top)
```

Example:
`stack = []`
`push(stack, 10)` → [10]
`push(stack, 20)` → [10, 20]
`push(stack, 30)` → [10, 20, 30]

Pop operation with underflow check

```
def pop(stack):
    if len(stack) == 0:
        print("Underflow")
        return None
    return stack.pop()   # removes & returns last element
```

Example:
`stack = [10, 20, 30]`
`pop(stack)` → 30 (stack becomes [10, 20])
`pop(stack)` → 20 (stack becomes [10])
`pop(stack)` → 10 (stack becomes [])
`pop(stack)` → prints **"Underflow"**, returns None

isEmpty operation

```
def isEmpty(stack):
    return len(stack) == 0
```

Display all elements (without deleting)

```
def display_all(stack):
    if len(stack) == 0:
        print("Empty Stack")
    else:
        for item in stack:
            print(item)
```

---

4. Important Patterns from CBSE Sample Papers

Push with tuples (stack of records)

```
def push_Clr(ClrStack, new_Clr):
    ClrStack.append(new_Clr)   # new_Clr can be a tuple
```

Example record: `('Yellow', 237, 250, 68)`

Pop with underflow message

```
def pop_Clr(ClrStack):
    if len(ClrStack) == 0:
        print("Underflow")
        return None
    return ClrStack.pop()
```

Push last 5 elements from a list to stack

```
def push_trail(N, myStack):
    # N is a list, myStack is a stack (list)
    for i in range(len(N)-5, len(N)):
        myStack.append(N[i])
```

Display stack elements

```
def display_all(myStack):
    if len(myStack) == 0:
        print("Empty Stack")
    else:
        for item in myStack:
            print(item)
```

---

5. Real-World Applications

- **Browser History / Back button** – URLs are pushed when visiting; back button pops the top URL.
- **Function Call Management** – function calls are managed using stack memory.
- **Undo operations** in text editors – actions stored in stack for undo.

---

6. Stack Overflow and Underflow

- **Stack Overflow** – occurs when we try to **push** into a **full** stack (capacity exceeded).
- **Stack Underflow** – occurs when we try to **pop** from an **empty** stack.

> ✅ **Always check underflow before pop:**
> `if len(stack) == 0:`
> `print("Underflow")`

---

7. Quick Revision Table

Operation	Python Method	Description
Push	`stack.append(item)`	Adds element to the top
Pop	`stack.pop()`	Removes and returns top element
Check Empty	`len(stack) == 0`	Returns `True` if empty
Check Size	`len(stack)`	Returns number of elements

---

8. Common Mistakes to Avoid

- Forgetting underflow check before `pop()`.
- Not using `append()` for push.
- Confusing `pop()` with removing a specific element.
- Accessing elements without checking stack length.

---

9. Complete Working Example

```
# Full stack implementation
stack = []

def push(stack, item):
    stack.append(item)
    print("Pushed:", item)

def pop(stack):
    if len(stack) == 0:
        print("Underflow")
        return None
    removed = stack.pop()
    print("Popped:", removed)
    return removed
```

```
def display(stack):
    if len(stack) == 0:
        print("Empty Stack")
    else:
        print("Stack:", stack)

# Usage
push(stack, "Laptop")
push(stack, "Mobile")
push(stack, "Headphones")
display(stack)          # Stack: ['Laptop', 'Mobile', 'Headphones']
pop(stack)              # Popped: Headphones
display(stack)          # Stack: ['Laptop', 'Mobile']
pop(stack)              # Popped: Mobile
pop(stack)              # Popped: Laptop
pop(stack)              # Underflow
```

10. Important Terms to Remember

LIFO	Last In First Out
STACK	linear LIFO data structure
PUSH	add element to top
POP	remove element from top
OVERFLOW	push on full stack
UNDERFLOW	pop from empty stack
PEEK	view top without removing
TOP	most recently added element